ORBIT · PYTHON · PAPER 03

第 3 套

59 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 14

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（4）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

关于Python的私有属性，以下说法正确的是：

A. 使用单下划线开头定义私有属性
B. 使用双下划线开头定义私有属性
C. Python有严格的私有属性限制
D. 私有属性完全无法访问

答案 B. 使用双下划线开头定义私有属性

解析 Python中的名称改编（Name Mangling）机制。
示例：

```
class MyClass:
    def __init__(self):
        self._single = "protected"
        # 约定为保护
        self.__double = "private"
    # 名称改编
    def get_private(self):
        return self.__double
obj = MyClass()
print(obj._single)
# protected
# print(obj.__double) # AttributeError
print(obj._MyClass__double)
# private - 名称改编后的名称
print(obj.get_private())
# private
```

知识拓展：

- 单下划线 _ ：约定为内部使用，但不强制
- 双下划线 __ ：触发名称改编，变为 _ClassName__attribute
- 名称改编主要用于避免子类意外覆盖父类属性

02 • 单项选择

以下代码的输出是什么？

```
class A:
    def __init__(self):
        self.__x = 10
a = A()
print(a._A__x)
```

A. 报错
B. 10
C. None
D. __x

答案 B. 10

解析 __new__() 与 __init__() 的区别：

- __new__() ：真正的构造函数，负责创建并返回实例对象
- __init__() ：初始化函数，对已创建的实例进行初始化

• __new__() 先执行， __init__() 后执行

示例：

```
class MyClass:
    def __new__(cls, *args, **kwargs):
        print("1. __new__被调用，创建实例")
        instance = super().__new__(cls)
        return instance
    def __init__(self, value):
        print("2. __init__被调用，初始化实例")
        self.value = value
obj = MyClass(42)
# 输出：
# 1. __new__被调用，创建实例
# 2. __init__被调用，初始化实例
```

知识拓展： __new__() 的主要应用场景：

1. 实现单例模式
2. 继承不可变类型（如int、str、tuple）
3. 元类编程

```
# 单例模式示例
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
s1 = Singleton()
s2 = Singleton()
print(s1 is s2)
# True - 同一个实例
```

03 • 单项选择

关于类的 __new__() 方法，以下说法正确的是：

A. 在 __init__() 之后调用

B. 在 __init__() 之前调用

C. 用于初始化实例

D. 不能被重写

答案 B. 在 __init__() 之前调用

解析 上下文管理器协议包含两个核心方法：

• __enter__() ：进入with块时调用，返回值赋给as后的变量

• __exit__(exc_type, exc_val, exc_tb) ：退出with块时调用，处理异常

示例：

```
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
        self.file = None
    def __enter__(self):
        print(f"打开文件: {self.filename}")
        self.file = open(self.filename, self.mode)
        return self.file
```

```
    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"关闭文件: {self.filename}")
        if self.file:
            self.file.close()
        if exc_type:
            print(f"异常类型: {exc_type}")
        return False
# 不抑制异常
with FileManager('test.txt', 'w') as f:
    f.write('Hello, World!')
# 输出:
# 打开文件: test.txt
# 关闭文件: test.txt
```

知识拓展： __exit__() 返回True可以抑制异常，False则继续抛出异常。

04 • 单项选择

以下哪个魔法方法用于实现对象的上下文管理？

A. __enter__() 和 __exit__()
B. __open__() 和 __close__()
C. __start__() 和 __end__()
D. __begin__() 和 __finish__()

答案 A. __enter__() 和 __exit__()

解析 Python使用C3线性化算法（MRO - Method Resolution Order）解决多重继承的方法查找顺序。

示例：

```
class A:
    def method(self):
        print("A")
class B(A):
    def method(self):
        print("B")
class C(A):
    def method(self):
        print("C")
class D(B, C):
    # 多重继承
    pass
d = D()
d.method()
# 输出: B
# 查看MRO
print(D.__mro__)
# (<class '__main__.D'>, <class '__main__.B'>,
# <class '__main__.C'>, <class '__main__.A'>,
# <class 'object'>)
```

知识拓展：MRO遵循三个原则：

1. 子类优先于父类
2. 多个父类按照定义顺序
3. 保持单调性（如果C在A之前，则所有子类中C都在A之前）

05 • 单项选择

关于Python的MRO（方法解析顺序），以下说法正确的是：

A. 使用深度优先搜索
B. 使用广度优先搜索
C. 使用C3线性化算法
D. 随机选择

答案 C. 使用C3线性化算法

解析 super() 用于调用父类方法，在多重继承中遵循MRO顺序。

示例：

```
class A:
    def method(self):
        print("A.method")
        return "A"
class B(A):
    def method(self):
        print("B.method")
        result = super().method()
        # 调用A.method
        return "B" + result
class C(A):
    def method(self):
        print("C.method")
        result = super().method()
        # 调用A.method
        return "C" + result
class D(B, C):
    def method(self):
        print("D.method")
        result = super().method()
        # 按MRO调用B.method
        return "D" + result
d = D()
result = d.method()
# 输出：
# D.method
# B.method
# C.method
# A.method
print(result)
# DBCA
```

知识拓展： super() 的优点：

1. 避免硬编码父类名
2. 支持协作式多重继承
3. 遵循MRO顺序

06 • 单项选择

以下代码的输出是什么？

```
class A:
    x = 1
```

```
class B(A):
    pass
class C(A):
    x = 2
class D(B, C):
    pass
print(D.x)
```

A. 1
B. 2
C. 报错
D. None

答案 B. 2

解析 根据MRO查找顺序，D的方法解析顺序是D->B->C->A。
示例：

```
class A:
    x = 1
class B(A):
    pass
class C(A):
    x = 2
class D(B, C):
    pass
print(D.x)
# 2
print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
# 按MRO顺序查找x：D没有 -> B没有 -> C有x=2 -> 返回2
```

知识拓展：类属性查找遵循MRO顺序，实例属性查找先查实例字典，再查类属性。

07 • 单项选择

关于生成器的 send() 方法，以下说法正确的是：

A. 只能发送None
B. 可以向生成器发送值
C. 与 next() 完全相同
D. 不能使用

答案 B. 可以向生成器发送值

解析 send() 方法可以向生成器发送值，并恢复生成器执行。
示例：

```
def generator():
    print("生成器启动")
    while True:
        value = yield
        # 接收send()发送的值
        print(f"收到值: {value}")
gen = generator()
next(gen)
# 启动生成器，输出：生成器启动
gen.send(10)
```

```
# 发送10，输出：收到值: 10
gen.send(20)
# 发送20，输出：收到值: 20
```

知识拓展：生成器的三个主要方法：

1. `__next__()` 或 `next()` ：获取下一个值
2. `send(value)` ：发送值到生成器
3. `close()` ：关闭生成器

08 • 单项选择

以下代码的输出是什么？

```
def gen():
    x = yield 1
    yield x
g = gen()
print(next(g))
print(g.send(10))
```

A. 1 10
B. 1 None
C. None 10
D. 报错

答案 A. 1 10

解析 生成器中 yield 表达式的返回值来自 send() 方法。

示例：

```
def gen():
    x = yield 1
    # 第1次yield返回1，等待接收值
    print(f"x = {x}")
    y = yield x + 2
    # 第2次yield返回x+2
    print(f"y = {y}")
g = gen()
print(next(g))
# 1 - 第1次yield返回1
print(g.send(10))
# x = 10，输出12 - send(10)赋值给x，第2次yield返回12
```

知识拓展：协程的工作原理：

- yield 暂停函数执行并返回值
- send() 恢复执行并传入值
- 第一次必须用 next() 或 send(None) 启动

09 • 单项选择

关于迭代器的 `__iter__()` 方法，以下说法正确的是：

A. 必须返回self
B. 必须返回新的迭代器
C. 可以返回任何可迭代对象
D. 不需要返回值

答案 A. 必须返回self

解析 迭代器协议要求 `__iter__()` 返回迭代器对象本身。

示例：

```
class MyIterator:
    def __init__(self, max_num):
        self.max_num = max_num
        self.current = 0
    def __iter__(self):
        return self
    # 返回自身
    def __next__(self):
        if self.current < self.max_num:
            self.current += 1
            return self.current
        raise StopIteration
my_iter = MyIterator(3)
for num in my_iter:
    print(num)
# 1, 2, 3
```

知识拓展：可迭代对象vs迭代器：

- 可迭代对象：实现 `__iter__()` 方法
- 迭代器：实现 `__iter__()` 和 `__next__()` 方法

10 • 单项选择

以下哪个不是有效的装饰器语法？

A. `@decorator`

B. `@decorator()`

C. `@decorator(arg)`

D. `decorator@`

答案 D. `decorator@`

解析 装饰器的正确语法是 `@decorator_name` ， `decorator@` 不是有效语法。

示例：

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("函数执行前")
        result = func(*args, **kwargs)
        print("函数执行后")
        return result
    return wrapper
@my_decorator
# 正确语法
def say_hello():
    print("Hello!")
say_hello()
# 输出：
# 函数执行前
# Hello!
# 函数执行后
```

知识拓展：装饰器本质是语法糖：

```
@decorator
def func():
```

```
    pass
# 等价于
def func():
    pass
func = decorator(func)
```

11 • 单项选择

关于装饰器的执行顺序，以下说法正确的是：

```
@decorator1
@decorator2
def func():
    pass
```

A. 先执行decorator1，再执行decorator2

B. 先执行decorator2，再执行decorator1

C. 同时执行

D. 不确定

答案 B. 先执行decorator2，再执行decorator1

解析 多个装饰器的执行顺序是从下到上（靠近函数定义的先执行），但调用顺序是从上到下。

示例：

```
def decorator1(func):
    print("装饰器1执行")
    def wrapper(*args, **kwargs):
        print("装饰器1调用")
        return func(*args, **kwargs)
    return wrapper
def decorator2(func):
    print("装饰器2执行")
    def wrapper(*args, **kwargs):
        print("装饰器2调用")
        return func(*args, **kwargs)
    return wrapper
@decorator1
@decorator2
def my_func():
    print("原函数")
# 输出：
# 装饰器2执行 <- 从下到上执行
# 装饰器1执行
my_func()
# 输出：
# 装饰器1调用 <- 从上到下调用
# 装饰器2调用
# 原函数
```

知识拓展：装饰器等价形式：

```
@decorator1
@decorator2
def func():
    pass
```

```
# 等价于
func = decorator1(decorator2(func))
```

12 • 单项选择

以下代码的输出是什么？

```
def outer():
    x = []
    def inner(y):
        x.append(y)
        return x
    return inner
f = outer()
print(f(1))
print(f(2))
```

A. [1] [2]

B. [1] [1, 2]

C. [1, 2] [1, 2]

D. 报错

答案 B. [1] [1, 2]

解析 闭包中的自由变量（外层函数的变量）会被保存，所有闭包共享同一个变量对象。

示例：

```
def create_multipliers():
    multipliers = []
    for i in range(3):
        def multiplier(x):
            return x * i
        # i是自由变量
        multipliers.append(multiplier)
    return multipliers
funcs = create_multipliers()
print(funcs[0](2))
# 4 - 不是0，因为i=2
print(funcs[1](2))
# 4 - 所有闭包共享同一个i
print(funcs[2](2))
# 4
# 正确写法：使用默认参数捕获当前值
def create_multipliers_correct():
    multipliers = []
    for i in range(3):
        def multiplier(x, i=i):
            # i=i捕获当前值
            return x * i
        multipliers.append(multiplier)
    return multipliers
funcs = create_multipliers_correct()
print(funcs[0](2))
# 0
print(funcs[1](2))
```

```
# 2
print(funcs[2](2))
# 4
```

知识拓展：闭包的三个特性：

1. 内部函数引用外部函数的变量
2. 外部函数返回内部函数
3. 变量在外部函数结束后仍然存在

13 • 单项选择

关于 nonlocal 关键字，以下说法正确的是：

A. 用于声明全局变量

B. 用于声明外层函数的变量

C. 用于声明局部变量

D. 与 global 完全相同

答案 B. 用于声明外层函数的变量

解析 nonlocal 用于在内层函数中修改外层函数的变量。

示例：

```
def outer():
    count = 0
    def increment_wrong():
        count = count + 1
    # UnboundLocalError
    def increment_correct():
        nonlocal count
        # 声明使用外层变量
        count = count + 1
    increment_correct()
    print(count)
    # 1
    increment_correct()
    print(count)
# 2
outer()
```

对比global和nonlocal：

```
x = 0
# 全局变量
def outer():
    x = 10
    # 外层函数变量
    def use_global():
        global x
        x = 100
    # 修改全局变量
    def use_nonlocal():
        nonlocal x
        x = 20
    # 修改外层函数变量
    use_nonlocal()
    print(f"外层x: {x}")
# 20
```

```
outer()
print(f"全局x: {x}")
# 0
```

知识拓展： nonlocal 只能用于嵌套函数， global 用于全局变量。

14 • 单项选择

以下代码的输出是什么？

```
def func(**kwargs):
    return len(kwargs)
print(func(a=1, b=2, c=3))
```

A. 1

B. 2

C. 3

D. 6

答案 C. 3

解析 **kwargs 接收关键字参数，返回字典， len(kwargs) 是键值对的数量。

示例：

```
def func(**kwargs):
    print(f"kwargs类型: {type(kwargs)}")
    print(f"kwargs内容: {kwargs}")
    return len(kwargs)
result = func(a=1, b=2, c=3)
print(f"长度: {result}")
# 输出：
# kwargs类型: <class 'dict'>
# kwargs内容: {'a': 1, 'b': 2, 'c': 3}
# 长度: 3
```

完整的参数类型示例：

```
def full_params(pos1, pos2, *args, kw1, kw2=None, **kwargs):
    print(f"位置参数: {pos1}, {pos2}")
    print(f"可变位置参数: {args}")
    print(f"关键字参数: {kw1}, {kw2}")
    print(f"可变关键字参数: {kwargs}")
full_params(1, 2, 3, 4, kw1='a', kw2='b', extra1='c', extra2='d')
# 输出：
# 位置参数: 1, 2
# 可变位置参数: (3, 4)
# 关键字参数: a, b
# 可变关键字参数: {'extra1': 'c', 'extra2': 'd'}
```

知识拓展：参数顺序：位置参数 > *args > 关键字参数 > **kwargs

15 • 单项选择

关于参数解包，以下代码的输出是什么？

```
def func(a, b, c):
    return a + b + c
args = [1, 2, 3]
print(func(*args))
```

A. [1, 2, 3]

B. 6
C. 报错
D. 123

答案 B. 6

解析 *args 可以解包列表，将列表元素作为独立参数传递。
示例：

```
def sum_numbers(*args):
    print(f"args: {args}")
    return sum(args)
# 直接传递多个参数
result1 = sum_numbers(1, 2, 3)
print(result1)
# 6
# 使用*解包列表
numbers = [1, 2, 3]
result2 = sum_numbers(*numbers)
# 等价于sum_numbers(1, 2, 3)
print(result2)
# 6
# 对比：不解包
result3 = sum_numbers(numbers)
# 传递一个列表对象
print(result3)
# TypeError: unsupported operand type(s)
```

解包的各种用法：

```
# 函数调用时解包
def func(a, b, c):
    return a + b + c
values = [1, 2, 3]
print(func(*values))
# 6
# 列表解包
a, *b, c = [1, 2, 3, 4, 5]
print(a, b, c)
# 1 [2, 3, 4] 5
# 字典解包
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, **dict1}
print(dict2)
# {'c': 3, 'a': 1, 'b': 2}
```

知识拓展： * 用于解包序列， ** 用于解包字典。

16 • 单项选择

Pandas中，以下哪个方法用于数据透视？

A. pivot()
B. rotate()
C. transform()
D. reshape()

答案 A. pivot()

解析 pivot() 方法用于数据透视，将长格式数据转换为宽格式。

示例：

```
import pandas as pd
# 原始数据（长格式）
df = pd.DataFrame({
    'date': ['2023-01', '2023-01', '2023-02', '2023-02'],
    'product': ['A', 'B', 'A', 'B'],
    'sales': [100, 150, 120, 180]
})
# 数据透视
pivot_df = df.pivot(index='date', columns='product', values='sales')
print(pivot_df)
# product A B
# date
# 2023-01 100 150
# 2023-02 120 180
pivot vs pivot_table:
# pivot - 不支持聚合，要求索引唯一
pivot_df = df.pivot(index='date', columns='product', values='sales')
# pivot_table - 支持聚合函数
pivot_table_df = df.pivot_table(
    index='date',
    columns='product',
    values='sales',
    aggfunc='mean'
# 可以使用聚合函数
)
知识拓展：常用的数据重塑方法：
• pivot() ：简单透视
• pivot_table() ：带聚合的透视
• melt() ：宽格式转长格式
• stack()/unstack() ：索引级别转换
```

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5])
print(s[s > 2].index.tolist())
```

A. [3, 4, 5]

B. [2, 3, 4]

C. [0, 1, 2]

D. [1, 2]

答案 B. [2, 3, 4]

解析 loc[2:4] 在Pandas中是闭区间，包含起始和结束标签。

示例：

```
import pandas as pd
df = pd.DataFrame(
    {'A': [10, 20, 30, 40, 50, 60]},
    index=[0, 1, 2, 3, 4, 5]
```

```
)
# loc使用标签，闭区间
print(df.loc[2:4])
# A
# 2 30
# 3 40
# 4 50 <- 包含4
# iloc使用位置，左闭右开
print(df.iloc[2:4])
# A
# 2 30
# 3 40 <- 不包含4
loc vs iloc对比：
        对比：
df = pd.DataFrame(
    {'A': [10, 20, 30]},
    index=['a', 'b', 'c']
)
# loc - 基于标签
print(df.loc['a':'c'])
# 包含'c'
# iloc - 基于位置
print(df.iloc[0:2])
# 不包含位置2
知识拓展：Pandas的索引规则：
• loc ：基于标签，闭区间
• iloc ：基于位置，左闭右开
• at ：快速标签访问单个值
• iat ：快速位置访问单个值
```

18 • 单项选择

Pandas中， inplace=True 参数的作用是：

A. 创建新的DataFrame

B. 在原DataFrame上修改

C. 不做任何修改

D. 删除DataFrame

答案 B. 在原DataFrame上修改

解析 inplace=True 直接在原DataFrame上修改，不返回新对象。

```
示例：
import pandas as pd
df = pd.DataFrame({'A': [3, 1, 2]})
# inplace=False（默认）：返回新对象
df_new = df.sort_values('A')
print("原df:")
print(df)
# A列仍是[3, 1, 2]
print("新df:")
print(df_new)
# A列是[1, 2, 3]
# inplace=True：修改原对象
```

```
df.sort_values('A', inplace=True)
print("修改后的df:")
print(df)
# A列变为[1, 2, 3]
支持inplace的常用方法：
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
# 删除列
df.drop('B', axis=1, inplace=True)
# 填充缺失值
df.fillna(0, inplace=True)
# 排序
df.sort_values('A', inplace=True)
# 重置索引
df.reset_index(drop=True, inplace=True)
知识拓展：使用 inplace=True 的注意事项：
1. 节省内存（不创建副本）
2. 不能链式调用（返回None）
3. Pandas 2.0+推荐避免使用inplace
```

19 • 单项选择

关于Pandas的 merge() 方法，以下说法正确的是：

A. 只能按索引合并

B. 只能按列名合并

C. 可以指定合并方式（inner、outer等）

D. 不能合并多个DataFrame

答案 C. 可以指定合并方式（inner、outer等）

解析 merge() 方法可以通过 how 参数指定不同的合并方式。

```
示例：
import pandas as pd
df1 = pd.DataFrame({'key': ['A', 'B', 'C'], 'val1': [1, 2, 3]})
df2 = pd.DataFrame({'key': ['B', 'C', 'D'], 'val2': [4, 5, 6]})
# inner join（默认）：保留两个表都有的键
inner = pd.merge(df1, df2, on='key', how='inner')
print("Inner:")
print(inner)
# 只有B和C
# outer join：保留所有键
outer = pd.merge(df1, df2, on='key', how='outer')
print("Outer:")
print(outer)
# A, B, C, D，缺失值为NaN
# left join：保留左表所有键
left = pd.merge(df1, df2, on='key', how='left')
print("Left:")
print(left)
# A, B, C
# right join：保留右表所有键
right = pd.merge(df1, df2, on='key', how='right')
print("Right:")
```

```
print(right)
# B, C, D
```

merge的高级用法：

```
# 多列合并
df1 = pd.DataFrame({'key1': ['A', 'B'], 'key2': [1, 2], 'val1': [10, 20]})
df2 = pd.DataFrame({'key1': ['A', 'B'], 'key2': [1, 2], 'val2': [30, 40]})
result = pd.merge(df1, df2, on=['key1', 'key2'])
# 不同列名
df1 = pd.DataFrame({'left_key': ['A', 'B'], 'val1': [1, 2]})
df2 = pd.DataFrame({'right_key': ['A', 'B'], 'val2': [3, 4]})
result = pd.merge(df1, df2, left_on='left_key', right_on='right_key')
```

知识拓展：合并方法对比：

- merge() ：SQL风格的连接
- join() ：基于索引的连接
- concat() ：简单的拼接

20 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
print(df.shape)
```

A. (2, 2)

B. [2, 2]

C. 4

D. (4,)

答案 A. (2, 2)

解析 shape 属性返回(行数, 列数)的元组。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
print(df.shape)
# (3, 3) - 3行3列
print(type(df.shape))
# <class 'tuple'>
# 获取行数和列数
rows, cols = df.shape
print(f"行数: {rows}, 列数: {cols}")
# 也可以单独获取
print(f"行数: {len(df)}")
print(f"列数: {len(df.columns)}")
```

DataFrame的常用属性：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4.0, 5.0, 6.0],
```

```
    'C': ['x', 'y', 'z']
})
print(df.shape)
# (3, 3) - 形状
print(df.size)
# 9 - 总元素数
print(df.ndim)
# 2 - 维度
print(df.columns)
# Index(['A', 'B', 'C']) - 列名
print(df.index)
# RangeIndex(start=0, stop=3) - 索引
print(df.dtypes)
# 各列数据类型
print(df.values)
# numpy数组形式
```

知识拓展：DataFrame的维度信息：

- shape ：返回(行, 列)元组
- size ：返回总元素数（行×列）
- ndim ：返回维度数（DataFrame总是2）
- len(df) ：返回行数

21 • 单项选择

pathlib.Path 对象的 with_suffix() 方法的作用是：

A. 删除文件扩展名
B. 替换文件扩展名
C. 添加文件扩展名
D. 获取文件扩展名

答案 B. 替换文件扩展名

解析 with_suffix() 用于更改文件扩展名，返回新的Path对象。

示例：

```
from pathlib import Path
p = Path('/home/user/docs/file.txt')
# 更改扩展名
new_p = p.with_suffix('.md')
print(new_p)
# /home/user/docs/file.md
# 删除扩展名
no_suffix = p.with_suffix('')
print(no_suffix)
# /home/user/docs/file
# 更改多重扩展名
p2 = Path('archive.tar.gz')
new_p2 = p2.with_suffix('.zip')
# 只更改最后一个
print(new_p2)
# archive.tar.zip
```

知识拓展：Path对象的修改方法：

- with_suffix() ：更改扩展名
- with_name() ：更改文件名

- with_stem() ：Python 3.9+，更改主文件名

22 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('file.txt')
print(p.with_suffix('.md'))
```

A. file.txt.md

B. file.md

C. .md

D. file

答案 B. file.md

解析 with_suffix() 可以将.txt扩展名替换为.md。

示例：

```
from pathlib import Path
# 单个文件
original = Path('README.txt')
markdown = original.with_suffix('.md')
print(markdown)
# README.md
# 批量转换
import os
for file in Path('.').glob('*.txt'):
    new_file = file.with_suffix('.md')
    # os.rename(file, new_file) # 实际重命名
    print(f"{file} -> {new_file}")
```

知识拓展：文件名组件：

```
from pathlib import Path
p = Path('/home/user/docs/file.txt')
print(p.name)
# file.txt - 完整文件名
print(p.stem)
# file - 主文件名
print(p.suffix)
# .txt - 扩展名
print(p.suffixes) # ['.txt'] - 所有扩展名
```

23 • 单项选择

Path.iterdir() 方法返回的是：

A. 列表

B. 元组

C. 生成器

D. 字典

答案 C. 生成器

解析 iterdir() 返回生成器，需要迭代或转换为列表。

示例：

```
from pathlib import Path
project_dir = Path('/home/user/project')
```

```
# iterdir()返回生成器
entries = project_dir.iterdir()
print(type(entries))
# <class 'generator'>
# 迭代使用
for entry in project_dir.iterdir():
    if entry.is_file():
        print(f"文件: {entry.name}")
    elif entry.is_dir():
        print(f"目录: {entry.name}")
# 转换为列表
all_entries = list(project_dir.iterdir())
print(f"总共{len(all_entries)} 个条目")
# 筛选文件
files = [f for f in project_dir.iterdir() if f.is_file()]
dirs = [d for d in project_dir.iterdir() if d.is_dir()]
```

知识拓展：目录遍历方法对比：

- iterdir() ：返回当前目录的直接子项
- glob(pattern) ：模式匹配，非递归
- rglob(pattern) ：模式匹配，递归
- walk() ：Python 3.12+，类似os.walk()

24 • 单项选择

关于 Path.resolve() 方法，以下说法正确的是：

A. 删除路径

B. 解析为绝对路径

C. 创建路径

D. 检查路径

答案 B. 解析为绝对路径

解析 resolve() 解析符号链接并返回绝对路径。

示例：

```
from pathlib import Path
import os
# 相对路径
relative = Path('docs/readme.md')
print(relative)
# docs/readme.md
# 转为绝对路径
absolute = relative.resolve()
print(absolute)
# /home/user/project/docs/readme.md
# 解析符号链接
os.symlink('/original/path', 'symlink')
symlink_path = Path('symlink')
resolved = symlink_path.resolve()
print(resolved)
# /original/path - 真实路径
# 简化路径
complex_path = Path('/home/user/../user/./docs')
simplified = complex_path.resolve()
```

```
print(simplified)
# /home/user/docs
```

知识拓展：路径解析方法：

- resolve() ：完全解析，返回绝对路径
- absolute() ：转为绝对路径，不解析符号链接
- expanduser() ：展开~为用户主目录

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('a/b/../c')
print(p)
```

A. a/c

B. a/b/../c

C. c

D. 报错

答案 B. a/b/../c

解析 Path对象不会自动简化路径，需要调用 resolve() 。

示例：

```
from pathlib import Path
# 不自动简化
p1 = Path('/home/user/../user/docs')
print(p1)
# /home/user/../user/docs - 保持原样
# 使用resolve()简化
p2 = p1.resolve()
print(p2)
# /home/user/docs - 简化后
# ./也不会自动移除
p3 = Path('./docs/./file.txt')
print(p3)
# docs/./file.txt
print(p3.resolve())
# /home/user/project/docs/file.txt
# 多个..
p4 = Path('/a/b/c/../../d')
print(p4)
# /a/b/c/../../d
print(p4.resolve())
# /a/d
```

知识拓展：路径规范化的重要性：

1. 避免路径比较错误
2. 解析符号链接
3. 保证路径唯一性

26 • 单项选择

关于 map() 函数，以下说法正确的是：

A. 返回列表

B. 返回map对象（迭代器）

C. 只能接受一个可迭代对象

D. 不能使用lambda函数

答案 B. 返回map对象（迭代器）

解析 map() 返回map对象（迭代器），需要转换为列表或迭代。

示例：

```
numbers = [1, 2, 3, 4, 5]
# map()返回迭代器
result = map(lambda x: x * 2, numbers)
print(type(result))
# <class 'map'>
print(result)
# <map object at 0x...>
# 转换为列表
result_list = list(map(lambda x: x * 2, numbers))
print(result_list)
# [2, 4, 6, 8, 10]
# 直接迭代
for item in map(lambda x: x * 2, numbers):
    print(item)
# 2, 4, 6, 8, 10
# 使用函数名
def square(x):
    return x ** 2
squares = list(map(square, numbers))
print(squares)
# [1, 4, 9, 16, 25]
map vs 列表推导式：
numbers = [1, 2, 3, 4, 5]
# 使用map
result1 = list(map(lambda x: x * 2, numbers))
# 使用列表推导式
result2 = [x * 2 for x in numbers]
print(result1 == result2)
# True
# 列表推导式通常更Pythonic
知识拓展：函数式编程工具：
• map(func, iterable) ：映射
• filter(func, iterable) ：过滤
• reduce(func, iterable) ：聚合（functools模块）
```

27 • 单项选择

以下代码的输出是什么？

```
result = list(map(str, [1, 2, 3]))
print(result)
```

A. [1, 2, 3]

B. ['1', '2', '3']

C. '123'

D. 报错

答案 B. ['1', '2', '3']

解析 map(str, iterable) 将迭代器中的整数转换为字符串。

示例：

```
numbers = [1, 2, 3, 4, 5]
# 转换为字符串
str_numbers = list(map(str, numbers))
print(str_numbers)
# ['1', '2', '3', '4', '5']
# 常用场景：拼接字符串
result = ', '.join(map(str, numbers))
print(result)
# '1, 2, 3, 4, 5'
# 其他类型转换
str_list = ['1', '2', '3']
int_list = list(map(int, str_list))
print(int_list)
# [1, 2, 3]
float_list = list(map(float, ['1.5', '2.7', '3.9']))
print(float_list)
# [1.5, 2.7, 3.9]
常用的类型转换：
# str - 转字符串
list(map(str, [1, 2, 3]))
# ['1', '2', '3']
# int - 转整数
list(map(int, ['1', '2', '3']))
# [1, 2, 3]
# float - 转浮点数
list(map(float, [1, 2, 3]))
# [1.0, 2.0, 3.0]
# bool - 转布尔值
list(map(bool, [0, 1, '', 'a']))
# [False, True, False, True]
知识拓展： map() 可以接受多个可迭代对象：
list1 = [1, 2, 3]
list2 = [4, 5, 6]
# 多个参数
result = list(map(lambda x, y: x + y, list1, list2))
print(result)
# [5, 7, 9]
```

28 • 单项选择

关于 reduce() 函数，以下说法正确的是：

A. 在内置函数中

B. 在 functools 模块中

C. 在 itertools 模块中

D. 已被废弃

答案 B. 在 functools 模块中

解析 reduce() 函数在 functools 模块中，不是内置函数。

示例：

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
# 计算乘积
product = reduce(lambda x, y: x * y, numbers)
print(product)
# 120 - 1*2*3*4*5
# 计算和
sum_result = reduce(lambda x, y: x + y, numbers)
print(sum_result)
# 15
# 指定初始值
result = reduce(lambda x, y: x + y, numbers, 100)
print(result)
# 115 - 100+1+2+3+4+5
# 找最大值
max_value = reduce(lambda x, y: x if x > y else y, numbers)
print(max_value)
# 5
reduce的工作原理：
from functools import reduce
# reduce的执行过程
numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, numbers)
# 等价于：
# step1: 1 + 2 = 3
# step2: 3 + 3 = 6
# step3: 6 + 4 = 10
print(result)
# 10
常用场景：
from functools import reduce
# 字符串拼接
words = ['Hello', ' ', 'World']
sentence = reduce(lambda x, y: x + y, words)
print(sentence)
# 'Hello World'
# 嵌套列表展开
lists = [[1, 2], [3, 4], [5, 6]]
flat = reduce(lambda x, y: x + y, lists)
print(flat)
# [1, 2, 3, 4, 5, 6]
# 阶乘
import operator
factorial_5 = reduce(operator.mul, range(1, 6))
print(factorial_5)
# 120
知识拓展：Python 2中 reduce() 是内置函数，Python 3中移到了 functools 模块。
```

29 • 单项选择

以下代码的输出是什么？

```
from functools import reduce
result = reduce(lambda x, y: x * y, [1, 2, 3, 4])
```

```
print(result)
```

A. 10

B. 24

C. [1, 2, 3, 4]

D. 报错

答案 B. 24

解析 reduce() 计算乘积：123*4=24。

示例：

```
from functools import reduce
import operator
# 使用lambda
result1 = reduce(lambda x, y: x * y, [1, 2, 3, 4])
print(result1)
# 24
# 使用operator.mul
result2 = reduce(operator.mul, [1, 2, 3, 4])
print(result2)
# 24
# 阶乘函数
def factorial(n):
    return reduce(operator.mul, range(1, n + 1))
print(factorial(4))
# 24
print(factorial(5))
# 120
print(factorial(10)) # 3628800
reduce vs 循环：
        循环：
from functools import reduce
numbers = [1, 2, 3, 4]
# 使用reduce
result1 = reduce(lambda x, y: x * y, numbers)
# 使用循环
result2 = 1
for num in numbers:
    result2 *= num
print(result1 == result2)
# True
# reduce更简洁，但可读性略差
知识拓展：Python内置了一些常用聚合函数：
numbers = [1, 2, 3, 4, 5]
print(sum(numbers))
# 15 - 求和
print(max(numbers))
# 5 - 最大值
print(min(numbers))
# 1 - 最小值
print(all([True, True, False]))
# False - 全为True
print(any([False, False, True])) # True - 任一True
```

30 • 单项选择

关于 all() 和 any() 函数，以下说法正确的是：

A. all() 要求所有元素为True
B. any() 要求至少一个元素为True
C. 两者都接受可迭代对象
D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是Python函数式编程的特性。
示例：

```
# A. 高阶函数：函数作为参数或返回值
def apply_twice(func, value):
    return func(func(value))
result = apply_twice(lambda x: x * 2, 3)
print(result)
# 12 - (3*2)*2
# B. Lambda表达式：匿名函数
square = lambda x: x ** 2
print(square(5))
# 25
# C. 闭包：内部函数引用外部变量
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply
times_3 = multiplier(3)
print(times_3(4))
# 12
# D. 不可变性：避免副作用
original = [1, 2, 3]
# 不修改原列表，返回新列表
new_list = [x * 2 for x in original]
print(original)
# [1, 2, 3] - 未变
print(new_list)
# [2, 4, 6]
```

函数式编程的核心原则：

```
# 1. 纯函数：相同输入总是相同输出，无副作用
def pure_function(x, y):
    return x + y
# 不修改外部状态
# 2. 不可变性：不修改原数据
def add_element(lst, element):
    return lst + [element]
# 返回新列表
# 3. 高阶函数：函数作为一等公民
def compose(f, g):
    return lambda x: f(g(x))
# 4. 递归：函数调用自身
def factorial(n):
```

```
    return 1 if n <= 1 else n * factorial(n - 1)
```

知识拓展：Python支持函数式编程风格，但不是纯函数式语言：

- 支持：高阶函数、lambda、闭包、map/filter/reduce
- 不强制：不可变性、纯函数
- 混合范式：同时支持面向对象、过程式编程

二、不定项选择题

31 • 不定项选择

关于Python的元类（metaclass），以下说法正确的有：

A. 元类是创建类的类
B. 所有类的默认元类是 type
C. 可以自定义元类
D. 元类用于控制类的创建过程

答案 A. 元类是创建类的类；B. 所有类的默认元类是 type；C. 可以自定义元类；D. 元类用于控制类的创建过程

解析 所有选项都是封装的正确目的。

示例：

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance
    # A. 隐藏内部实现
    def deposit(self, amount):
        if amount > 0:
            # B. 提供访问控制
            self.__balance += amount
        else:
            raise ValueError("金额必须为正")
    def get_balance(self):
        # C. 提供简洁接口
        return self.__balance
    def transfer(self, other, amount):
        # D. 便于维护和修改
        if amount <= self.__balance:
            self.__balance -= amount
            other.deposit(amount)
account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())
# 1500
```

知识拓展：封装的三要素：

1. 数据隐藏：使用私有属性（ __ 前缀）
2. 接口提供：公共方法访问数据
3. 访问控制：验证和保护数据完整性

32 • 不定项选择

以下哪些是Python的描述符协议方法：

A. __get__()
B. __set__()

C. __delete__()

D. __desc__()

答案 A. __get__()；B. __set__()；C. __delete__()

解析 描述符协议包含 __get__ 、 __set__ 、 __delete__ 三个方法。

示例：

```
class Descriptor:
    def __init__(self, name):
        self.name = name
    def __get__(self, instance, owner):
        if instance is None:
            return self
        print(f"Getting {self.name}")
        return instance.__dict__.get(self.name, None)
    def __set__(self, instance, value):
        print(f"Setting {self.name} = {value}")
        if value < 0:
            raise ValueError("值不能为负")
        instance.__dict__[self.name] = value
    def __delete__(self, instance):
        print(f"Deleting {self.name}")
        del instance.__dict__[self.name]
class MyClass:
    value = Descriptor('value')
    def __init__(self, value):
        self.value = value
obj = MyClass(10)
print(obj.value)
# Getting value -> 10
obj.value = 20
# Setting value = 20
del obj.value
# Deleting value
```

知识拓展：描述符的应用场景：

- 属性验证和类型检查
- 延迟计算（lazy evaluation）
- 实现property、classmethod、staticmethod

33 • 不定项选择

关于Python的单例模式实现，以下方法正确的有：

A. 使用 __new__() 方法

B. 使用装饰器

C. 使用元类

D. 使用模块

答案 A. 使用 __new__() 方法；B. 使用装饰器；C. 使用元类；D. 使用模块

解析 所有方法都可以实现单例模式。

示例：

```
# A. 使用__new__方法
class Singleton1:
    _instance = None
```

```
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
# B. 使用装饰器
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance
@singleton
class Singleton2:
    pass
# C. 使用元类
class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]
class Singleton3(metaclass=SingletonMeta):
    pass
# D. 使用模块
# singleton_module.py
class Singleton4:
    pass
instance = Singleton4()
               # 模块只加载一次
# 验证
s1 = Singleton1()
s2 = Singleton1()
print(s1 is s2)
# True
```

知识拓展：单例模式的使用场景：

- 配置管理器
- 日志记录器
- 数据库连接池
- 线程池

34 • 不定项选择

关于生成器表达式和列表推导式，以下说法正确的有：

A. 生成器表达式使用 ()

B. 列表推导式使用 []

C. 生成器表达式更节省内存

D. 两者语法完全相同

答案 A. 生成器表达式使用 ()；B. 列表推导式使用 []；C. 生成器表达式更节省内存；D. 两者语法完全相同

解析 所有选项都是异常处理的正确做法。

示例：

```
# A. 捕获特定异常
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"除零错误: {e}")
# B. 使用finally清理资源
try:
    f = open('file.txt', 'r')
    data = f.read()
except FileNotFoundError:
    print("文件不存在")
finally:
    if 'f' in locals():
        f.close()
# 确保文件关闭
# C. 自定义异常
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"余额{balance}不足，需要{amount}")
def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount
try:
    withdraw(100, 150)
except InsufficientFundsError as e:
    print(e)
# D. 避免捕获所有异常（除非必要）
try:
    risky_operation()
except Exception as e:
    # 不推荐，太宽泛
    print(f"错误: {e}")
# 更好的做法：捕获具体异常
try:
    risky_operation()
except (ValueError, TypeError, KeyError) as e:
    print(f"预期的错误: {e}")
```

知识拓展：异常处理的最佳实践：

1. 捕获特定异常，避免裸except
2. 使用finally清理资源
3. 自定义异常提供更多上下文
4. 不要吞没异常（silent failure）

35 • 不定项选择

以下哪些操作会消耗迭代器：

A. list(iterator)

B. for item in iterator

C. next(iterator)

D. iter(iterator)

答案 A. list(iterator)；B. for item in iterator；C. next(iterator)

解析 iter() 返回迭代器本身，不消耗元素。

示例：

```
my_list = [1, 2, 3]
# A. iter()返回迭代器
iterator = iter(my_list)
print(type(iterator))
# <class 'list_iterator'>
# B. next()消耗元素
print(next(iterator))
# 1
print(next(iterator))
# 2
# C. 可以多次调用iter()，每次返回新迭代器
iter1 = iter(my_list)
iter2 = iter(my_list)
print(next(iter1))
# 1
print(next(iter2))
# 1 - 独立的迭代器
# D错误：iter()不会消耗元素
original = [1, 2, 3]
iterator = iter(original)
print(original)
# [1, 2, 3] - 原列表未变
迭代器vs可迭代对象：
# 可迭代对象：有__iter__方法
class Iterable:
    def __iter__(self):
        return iter([1, 2, 3])
# 迭代器：有__iter__和__next__方法
class Iterator:
    def __init__(self):
        self.data = [1, 2, 3]
        self.index = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.index < len(self.data):
            result = self.data[self.index]
            self.index += 1
            return result
        raise StopIteration
it = Iterator()
for item in it:
    print(item)
# 1, 2, 3
知识拓展：迭代器的优势：
```

- 节省内存（惰性求值）
- 支持无限序列
- 流式处理大数据

36 • 不定项选择

关于装饰器的应用，以下说法正确的有：

A. 可以用于函数
B. 可以用于类
C. 可以用于方法
D. 可以嵌套使用

答案 A. 可以用于函数；B. 可以用于类；C. 可以用于方法；D. 可以嵌套使用

解析 装饰器可以应用于函数、方法、类和生成器。

示例：

```
import time
# 通用装饰器
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__}耗时: {end-start:.4f}秒")
        return result
    return wrapper
# A. 装饰普通函数
@timer
def slow_function():
    time.sleep(0.1)
    return "完成"
slow_function()
# B. 装饰类方法
class MyClass:
    @timer
    def method(self):
        time.sleep(0.1)
        return "方法完成"
obj = MyClass()
obj.method()
# C. 装饰类
def add_str_method(cls):
    cls.__str__ = lambda self: f"{cls.__name__}实例"
    return cls
@add_str_method
class Person:
    pass
print(Person())
# Person实例
# D. 装饰生成器
@timer
def slow_generator():
    for i in range(3):
```

```
        time.sleep(0.01)
        yield i
for val in slow_generator():
    print(val)
```

知识拓展：装饰器的高级应用：

- 带参数的装饰器
- 类装饰器（使用 __call__ ）
- 多个装饰器叠加
- functools.wraps保留元数据

37 · 不定项选择

关于闭包的特性，以下说法正确的有：

A. 内部函数引用外部函数的变量

B. 外部函数返回内部函数

C. 可以保存状态

D. 不能修改外部变量

答案 A. 内部函数引用外部函数的变量；B. 外部函数返回内部函数；C. 可以保存状态

解析 使用 nonlocal 可以在闭包中修改外层变量。

示例：

```
# A. 闭包可以读取外层变量
def outer():
    count = 0
    def inner():
        print(count)
    # 可以读取
    inner()
outer()
# 0
# B. 使用nonlocal修改外层变量
def counter():
    count = 0
    def increment():
        nonlocal count
        # 声明修改外层变量
        count += 1
        return count
    return increment
count_func = counter()
print(count_func())
# 1
print(count_func())
# 2
# C. 闭包可以访问多层外部作用域
def level1():
    x = 1
    def level2():
        y = 2
        def level3():
            nonlocal x, y
            x += 1
```

```
            y += 1
            return x, y
        return level3
    return level2
func = level1()()
print(func())
                # (2, 3)
# D错误：没有nonlocal会创建局部变量
def wrong_example():
    count = 0
    def increment():
        count = count + 1
    # UnboundLocalError
    increment()
知识拓展：闭包的实际应用：
# 装饰器中的闭包
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
@repeat(3)
def say_hello():
    print("Hello!")
say_hello()
# 打印3次Hello!
```

38 • 不定项选择

以下哪些是Pandas的索引类型：

A. RangeIndex

B. Int64Index

C. DatetimeIndex

D. MultiIndex

答案 A. RangeIndex；B. Int64Index；C. DatetimeIndex；D. MultiIndex

解析 所有选项都是Pandas的索引类型。

示例：

```
import pandas as pd
# A. RangeIndex - 默认整数索引
df1 = pd.DataFrame({'A': [1, 2, 3]})
print(type(df1.index))
# RangeIndex
print(df1.index)
# RangeIndex(start=0, stop=3, step=1)
# B. Index - 通用索引
df2 = pd.DataFrame({'A': [1, 2, 3]}, index=['a', 'b', 'c'])
print(type(df2.index))
# Index
```

```
print(df2.index)
# Index(['a', 'b', 'c'], dtype='object')
# C. DatetimeIndex - 时间索引
dates = pd.date_range('2023-01-01', periods=3)
df3 = pd.DataFrame({'A': [1, 2, 3]}, index=dates)
print(type(df3.index))
# DatetimeIndex
print(df3.index)
# D. MultiIndex - 多级索引
arrays = [['A', 'A', 'B', 'B'], [1, 2, 1, 2]]
index = pd.MultiIndex.from_arrays(arrays, names=['letter', 'number'])
df4 = pd.DataFrame({'value': [10, 20, 30, 40]}, index=index)
print(type(df4.index))
# MultiIndex
print(df4)
```

更多索引类型：

```
import pandas as pd
# CategoricalIndex - 分类索引
df5 = pd.DataFrame(
    {'A': [1, 2, 3]},
    index=pd.CategoricalIndex(['low', 'medium', 'high'])
)
# IntervalIndex - 区间索引
intervals = pd.interval_range(start=0, end=3)
df6 = pd.DataFrame({'A': [1, 2, 3]}, index=intervals)
# PeriodIndex - 时期索引
periods = pd.period_range('2023-01', periods=3, freq='M')
df7 = pd.DataFrame({'A': [1, 2, 3]}, index=periods)
```

知识拓展：索引的作用：

- 快速数据访问和选择
- 自动对齐操作
- 支持时间序列分析
- 层次化数据组织

39 · 不定项选择

关于Pandas的 groupby() 操作，以下说法正确的有：

A. 可以按单列分组

B. 可以按多列分组

C. 返回GroupBy对象

D. 可以应用多种聚合函数

答案 A. 可以按单列分组；B. 可以按多列分组；C. 返回GroupBy对象；D. 可以应用多种聚合函数

解析 所有选项都Pandas DataFrame的正确操作方式。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
# A. 使用loc选择行和列
```

```
print(df.loc[0:1, ['A', 'B']])
# A B
# 0 1 4
# 1 2 5
# B. 使用iloc选择按位置
print(df.iloc[0:2, 0:2])
# A B
# 0 1 4
# 1 2 5
# C. 使用布尔索引
print(df[df['A'] > 1])
# A B C
# 1 2 5 8
# 2 3 6 9
# D. 使用query()方法
print(df.query('A > 1 and B < 7'))
# A B C
# 1 2 5 8
```

知识拓展：DataFrame选择方法对比：

- loc ：基于标签的选择
- iloc ：基于位置的选择
- [] ：布尔索引、列名选择
- query() ：使用字符串表达式

40 • 不定项选择

Pandas中，以下哪些方法用于数据重塑：

A. pivot()

B. melt()

C. stack()

D. unstack()

答案 A. pivot()；B. melt()；C. stack()；D. unstack()

解析 所有选项都是Pandas数据重塑方法。

示例：

```
import pandas as pd
# A. melt() - 宽转长
df_wide = pd.DataFrame({
    'id': [1, 2],
    '2023': [100, 200],
    '2024': [150, 250]
})
df_long = df_wide.melt(id_vars=['id'], var_name='year', value_name='sales')
print(df_long)
# id year sales
# 0 1 2023 100
# 1 2 2023 200
# 2 1 2024 150
# 3 2 2024 250
# B. pivot() - 长转宽
df_pivoted = df_long.pivot(index='id', columns='year', values='sales')
print(df_pivoted)
```

```
# C. stack() - 列转行
df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
stacked = df.stack()
print(stacked)
# 0 A 1
# B 2
# 1 A 3
# B 4
# D. unstack() - 行转列
unstacked = stacked.unstack()
print(unstacked)
# A B
# 0 1 2
# 1 3 4
```

知识拓展：数据重塑场景：

- melt ：制作时间序列数据
- pivot ：数据汇总分析
- stack/unstack ：处理多级索引

41 • 不定项选择

关于 pathlib 的路径操作，以下说法正确的有：

A. / 运算符用于路径拼接

B. joinpath() 方法用于拼接路径

C. 自动处理不同操作系统的路径分隔符

D. 支持相对路径和绝对路径

答案 A. / 运算符用于路径拼接；B. joinpath() 方法用于拼接路径；C. 自动处理不同操作系统的路径分隔符；D. 支持相对路径和绝对路径

解析 所有选项都是Pandas数据清洗的正确方法。

示例：

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'A': [1, 2, 2, np.nan, 5],
    'B': [10, 20, 20, 40, 50],
    'C': ['a', 'b', 'b', 'd', 'e']
})
# A. 删除重复行
df_no_dup = df.drop_duplicates()
print(df_no_dup)
# B. 处理缺失值
df_filled = df.fillna(0)
# 填充
df_dropped = df.dropna()
# 删除
print(df_filled)
# C. 替换值
df_replaced = df.replace({'a': 'A', 'b': 'B'})
print(df_replaced)
# D. 重命名列
df_renamed = df.rename(columns={'A': 'col_a', 'B': 'col_b'})
```

```
print(df_renamed.columns)
```

常用清洗操作：

```
import pandas as pd
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': ['25', '30', 'unknown'],
    'score': [85.5, 90.0, 88.3]
})
# 类型转换
df['age'] = pd.to_numeric(df['age'], errors='coerce')
# 删除空白
df['name'] = df['name'].str.strip()
# 统一大小写
df['name'] = df['name'].str.lower()
```

知识拓展：数据质量检查：

```
print(df.info())
# 数据类型和缺失值
print(df.describe())
# 描述性统计
print(df.isnull().sum())
# 缺失值计数
print(df.duplicated().sum())
# 重复行计数
```

42 · 不定项选择

Path 对象的以下哪些属性返回路径信息：

A. name
B. stem
C. suffix
D. parent

答案 A. name；B. stem；C. suffix；D. parent

解析 所有选项都返回Pathlib的路径信息。

示例：

```
from pathlib import Path
p = Path('/home/user/docs/project/file.txt')
# A. parent - 父目录
print(p.parent)
# /home/user/docs/project
print(p.parent.parent)
# /home/user/docs
# B. parents - 所有父目录
for parent in p.parents:
    print(parent)
# /home/user/docs/project
# C. name - 文件名
print(p.name)
# file.txt
# D. parts - 路径分量
print(p.parts)
# ('/', 'home', 'user', 'docs', 'project', 'file.txt')
```

```
更多路径信息属性：
from pathlib import Path
p = Path('/home/user/archive.tar.gz')
print(p.stem)
# archive.tar - 主文件名
print(p.suffix)
# .gz - 扩展名
print(p.suffixes)
# ['.tar', '.gz'] - 所有扩展名
print(p.anchor)
# / - 锚点（根目录）
print(p.drive)
# '' - 驱动器（Windows为C:）
知识拓展：路径操作与os.path的对比：
import os
from pathlib import Path
path_str = '/home/user/file.txt'
# os.path
print(os.path.dirname(path_str))
# /home/user
print(os.path.basename(path_str))
# file.txt
# pathlib (更面向对象)
p = Path(path_str)
print(p.parent)
# /home/user
print(p.name)
# file.txt
```

43 • 不定项选择

关于列表推导式的嵌套，以下说法正确的有：

A. 可以嵌套多层

B. 外层循环在前

C. 可以包含多个if条件

D. 可读性可能降低

答案 A. 可以嵌套多层；B. 外层循环在前；C. 可以包含多个if条件；D. 可读性可能降低

解析 所有选项都是列表推导式的正确特性。

示例：

```
# A. 更简洁 - 与for循环对比
# for循环
result = []
for i in range(5):
    if i % 2 == 0:
        result.append(i * 2)
# 列表推导式
result = [i * 2 for i in range(5) if i % 2 == 0]
print(result)
# [0, 4, 8]
# B. 支持条件筛选
squares = [x**2 for x in range(10) if x % 2 == 0]
```

```
print(squares)
# [0, 4, 16, 36, 64]
# C. 可以嵌套
matrix = [[i*j for j in range(3)] for i in range(3)]
print(matrix)
# [[0, 0, 0],
# [0, 1, 2],
# [0, 2, 4]]
# D. 一般比for循环快
import time
start = time.time()
result1 = [i**2 for i in range(10000)]
time1 = time.time() - start
start = time.time()
result2 = []
for i in range(10000):
    result2.append(i**2)
time2 = time.time() - start
print(f"推导式: {time1:.4f}s, 循环: {time2:.4f}s")
各种推导式：
# 列表推导式
list_comp = [x**2 for x in range(5)]
# 集合推导式
set_comp = {x**2 for x in range(5)}
# 字典推导式
dict_comp = {x: x**2 for x in range(5)}
# 生成器表达式
gen_exp = (x**2 for x in range(5))
知识拓展：推导式的复杂用法：
# 多重条件
result = [x for x in range(20) if x % 2 == 0 if x % 3 == 0]
print(result)
# [0, 6, 12, 18]
# 多重循环
result = [(x, y) for x in range(3) for y in range(3)]
print(result)
# [(0,0), (0,1), ..., (2,2)]
# if-else表达式
result = [x if x % 2 == 0 else -x for x in range(5)]
print(result)
# [0, -1, 2, -3, 4]
```

44 · 不定项选择

以下哪些是Python的函数式编程特性：

A. 高阶函数

B. lambda表达式

C. map/filter/reduce

D. 列表推导式

答案 A. 高阶函数；B. lambda表达式；C. map/filter/reduce；D. 列表推导式

解析 所有选项都是Python函数式编程的特性。

示例：

```
from functools import reduce
# A. map - 映射变换
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
print(squares)
# [1, 4, 9, 16, 25]
# B. filter - 过滤
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)
# [2, 4]
# C. reduce - 聚合
sum_all = reduce(lambda x, y: x + y, numbers)
print(sum_all)
# 15
# D. lambda - 匿名函数
add = lambda x, y: x + y
print(add(3, 5))
# 8
# 组合使用
result = reduce(
    lambda x, y: x + y,
    map(lambda x: x**2,
        filter(lambda x: x % 2 == 0, numbers))
)
print(result)
# 4 + 16 = 20
```

函数式编程风格对比：

```
numbers = [1, 2, 3, 4, 5]
# 命令式（过程式）
result = 0
for num in numbers:
    if num % 2 == 0:
        result += num ** 2
print(result)
# 20
# 函数式
result = sum(map(lambda x: x**2, filter(lambda x: x % 2 == 0, numbers)))
print(result)
# 20
# Pythonic（推导式）
result = sum(x**2 for x in numbers if x % 2 == 0)
print(result)
# 20
```

知识拓展：函数式编程的优缺点：

优点：

- 代码简洁、易读
- 更容易测试和调试
- 更容易并行化

缺点：

- 可能性能较差
- 学习曲线较陡
- 过度使用可能降低可读性

45 • 不定项选择

关于 zip() 函数的特性，以下说法正确的有：

A. 可以接受多个可迭代对象
B. 返回元组的迭代器
C. 以最短序列为准
D. 可以使用 zip(*zipped) 解压

答案 A. 可以接受多个可迭代对象；B. 返回元组的迭代器；C. 以最短序列为准

解析 Python是多范式语言，支持面向对象、过程式和函数式编程。

示例：

```
# A. 面向对象编程
class Calculator:
    def __init__(self):
        self.result = 0
    def add(self, x):
        self.result += x
        return self
    def get_result(self):
        return self.result
calc = Calculator()
result = calc.add(5).add(3).get_result()
print(result)
# 8
# B. 过程式编程
def calculate_total(numbers):
    total = 0
    for num in numbers:
        total += num
    return total
print(calculate_total([1, 2, 3]))
# 6
# C. 函数式编程
from functools import reduce
numbers = [1, 2, 3]
result = reduce(lambda x, y: x + y, numbers)
print(result)
# 6
```

三种范式对比：

```
# 同一任务：计算偶数平方和
data = [1, 2, 3, 4, 5]
# 1. 面向对象
class EvenSquareSum:
    def __init__(self, numbers):
        self.numbers = numbers
    def calculate(self):
        result = 0
        for num in self.numbers:
```

```
            if num % 2 == 0:
                result += num ** 2
        return result
obj = EvenSquareSum(data)
print(obj.calculate())
# 20
# 2. 过程式
def even_square_sum(numbers):
    result = 0
    for num in numbers:
        if num % 2 == 0:
            result += num ** 2
    return result
print(even_square_sum(data))
# 20
# 3. 函数式
from functools import reduce
result = reduce(
    lambda x, y: x + y,
    map(lambda x: x**2, filter(lambda x: x % 2 == 0, data))
)
print(result)
                # 20
# 4. Pythonic
result = sum(x**2 for x in data if x % 2 == 0)
print(result)
# 20
```

知识拓展：Python不是纯函数式语言的原因：

- 不强制不可变性
- 支持变量重赋值
- 有副作用（print、I/O操作）
- 不是惰性求值（默认是严格求值）

三、填空题

46 • 填空 / 简答

Python中，_____ 方法用于定义对象的字符串表示（面向用户），_____ 方法用于定义对象的官方字符串表示（面向开发者）。

答案 super() ， __mro__

解析 多重继承中的关键工具。

示例：

```
class A:
    def method(self):
        print("A")
class B(A):
    def method(self):
        super().method()
        # 调用父类方法
        print("B")
```

```
class C(A):
    def method(self):
        super().method()
        print("C")
class D(B, C):
    def method(self):
        super().method()
        print("D")
# 查看MRO
print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
D().method()
# A
# C
# B
# D
```

知识拓展：MRO使用C3线性化算法，保证继承的一致性。

47 • 填空 / 简答

在装饰器中，使用 _____ 模块的 _____ 装饰器可以保留原函数的元数据。

答案 with ， finally

解析 幂析：资源管理的两种方式。

示例：

```
# 使用with（上下文管理器）
with open('file.txt', 'r') as f:
    content = f.read()
# 文件自动关闭
# 使用finally
try:
    f = open('file.txt', 'r')
    content = f.read()
finally:
    f.close()
# 总是执行
```

知识拓展： with 更简洁，自动处理异常； finally 更灵活，但需要手动管理资源。

48 • 填空 / 简答

Pandas中，_____ 方法用于数据透视，_____ 方法用于将宽格式数据转换为长格式。

答案 fillna() ， dropna()

解析 幂析：Pandas处理缺失值的两种方法。

示例：

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [np.nan, 2, 3, 4],
    'C': [1, 2, 3, 4]
})
```

```
# fillna - 填充缺失值
df_filled = df.fillna(0)
df_filled = df.fillna(method='ffill')
# 前向填充
df_filled = df.fillna(df.mean())
# 用均值填充
# dropna - 删除缺失值
df_dropped = df.dropna()
# 删除包含缺失值的行
df_dropped = df.dropna(axis=1)
# 删除包含缺失值的列
```

知识拓展：可以使用 `isnull()` 、 `notnull()` 检查缺失值。

49 • 填空 / 简答

使用 _____ 函数可以创建计数器，使用 _____ 函数可以将多个迭代器组合。

答案 `filter()` ， `reduce()`

解析 幂析：函数式编程的两个核心函数。

示例：

```
from functools import reduce
# filter - 过滤元素
numbers = [1, 2, 3, 4, 5, 6]
even = list(filter(lambda x: x % 2 == 0, numbers))
print(even)
# [2, 4, 6]
# reduce - 累积计算
product = reduce(lambda x, y: x * y, numbers)
print(product)
                # 720 (1*2*3*4*5*6)
```

知识拓展： `reduce()` 在Python 3中移至 `functools` 模块。

50 • 填空 / 简答

`pathlib.Path` 对象的 _____ 方法用于读取文本文件，_____ 方法用于写入文本文件。

答案 `cwd()` ， `iterdir()`

解析 幂析：Path对象的目录操作方法。

示例：

```
from pathlib import Path
# cwd() - 获取当前工作目录
current = Path.cwd()
print(current)
# /home/user/project
# iterdir() - 迭代目录内容
for item in current.iterdir():
    if item.is_file():
        print(f"文件: {item.name}")
    elif item.is_dir():
        print(f"目录: {item.name}")
```

知识拓展： `iterdir()` 返回生成器，不递归。递归搜索使用 `rglob()` 。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
s1 = Singleton()
s2 = Singleton()
print(s1 is s2)
print(id(s1) == id(s2))
```

问题：

- (1) 两个 print 语句的输出分别是什么？
- (2) 这段代码实现了什么设计模式？
- (3) __new__() 方法的作用是什么？

答案 单例模式

- (1) 两个都输出： True
- (2) 单例模式（Singleton Pattern）
- (3) __new__() 用于创建实例，在 __init__() 之前调用，控制实例的创建过程

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment
c1 = counter()
c2 = counter()
print(c1())
print(c2())
```

问题：

- (1) 三个 print 语句的输出分别是什么？
- (2) nonlocal 关键字的作用是什么？
- (3) 为什么 c1 和 c2 的计数是独立的？

答案 • (1) 输出：1, 2, 1

- (2) nonlocal 声明要修改外层函数的变量，而不是创建新的局部变量
- (3) 每次调用 counter() 都创建新的闭包环境，有独立的 count 变量

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'category': ['A', 'B', 'A', 'B', 'A'],
    'value': [10, 20, 30, 40, 50]
})
result = df.groupby('category')['value'].agg(['sum', 'mean', 'count'])
print(result)
print(type(result))
```

问题：

- (1) result 的输出是什么样的？
- (2) agg() 方法可以接受哪些类型的参数？
- (3) 如何只获取A类别的统计信息？

答案 聚合

- (1) 输出DataFrame，包含sum、mean、count三列，索引为A和B
- (2) 可以接受函数名字符串、函数对象、函数列表、字典等
- (3) result.loc['A']

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
def process_files(directory):
    path = Path(directory)
    python_files = list(path.glob('**/*.py'))
    for file in python_files:
        size = file.stat().st_size
        yield {
            'name': file.name,
            'path': str(file),
            'size': size
        }
files = list(process_files('.'))
print(len(files))
```

问题：

- (1) glob('**/*.py') 中的 ** 表示什么？
- (2) stat().st_size 返回的是什么？
- (3) 这个函数为什么使用生成器？

答案 • (1) ** 表示递归搜索所有子目录

- (2) 返回文件大小（字节数）
- (3) 使用生成器可以节省内存，特别是处理大量文件时

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
from functools import wraps
def validate(func):
    @wraps(func)
    def wrapper(x):
        if x < 0:
            raise ValueError("参数必须非负")
        return func(x)
    return wrapper
@validate
def square_root(x):
    return x ** 0.5
try:
    print(square_root(16))
    print(square_root(-4))
except ValueError as e:
    print(e)
```

问题：

- (1) 完整的输出是什么？
- (2) @wraps(func) 的作用是什么？
- (3) 如果不使用装饰器，如何实现相同功能？

答案 • (1) 输出：

```
4.0
参数必须非负
```

- (2) @wraps(func) 保留原函数的元数据（如 __name__ 、 __doc__ 等）
- (3) 可以在函数内部添加验证逻辑，或者手动调用验证函数

解析

五、程序设计题

56 • 填空 / 简答

设计一个员工管理系统，包含以下类：

Employee类（基类）：

- 属性：姓名、工号、基本工资
- 方法： calculate_salary() （抽象方法）
- 魔法方法： __str__() 、 __eq__() （按工号比较）

Manager类（继承

类（继承Employee）：

）：

- 额外属性：奖金
- 重写 calculate_salary() ：基本工资 + 奖金

Developer类（继承

类（继承Employee）：

）：

- 额外属性：项目数量

- 重写 calculate_salary() ：基本工资 + 项目数量 * 1000

要求：

- 创建多个员工实例
- 使用列表推导式筛选工资大于10000的员工
- 实现 __repr__() 方法
- 使用 @property 实现只读属性

答案

```
from abc import ABC, abstractmethod
class Employee(ABC):
    def __init__(self, name, emp_id, base_salary):
        self._name = name
        self._emp_id = emp_id
        self._base_salary = base_salary
    @property
    def name(self):
        return self._name
    @property
    def emp_id(self):
        return self._emp_id
    @abstractmethod
    def calculate_salary(self):
        pass
    def __str__(self):
        return f"{self.name} (ID: {self.emp_id})"
    def __repr__(self):
        return f"{self.__class__.__name__}('{self.name}', '{self.emp_id}', {self._base_sal
    def __eq__(self, other):
        if isinstance(other, Employee):
            return self.emp_id == other.emp_id
        return False
class Manager(Employee):
    def __init__(self, name, emp_id, base_salary, bonus):
        super().__init__(name, emp_id, base_salary)
        self.bonus = bonus
    def calculate_salary(self):
        return self._base_salary + self.bonus
class Developer(Employee):
    def __init__(self, name, emp_id, base_salary, project_count):
        super().__init__(name, emp_id, base_salary)
        self.project_count = project_count
    def calculate_salary(self):
        return self._base_salary + self.project_count * 1000
# 使用示例
employees = [
    Manager("张三", "M001", 8000, 5000),
    Developer("李四", "D001", 7000, 3),
    Manager("王五", "M002", 9000, 3000),
    Developer("赵六", "D002", 6000, 5)
```

```
]
print("所有员工：")
for emp in employees:
    print(f"{emp} - 工资: {emp.calculate_salary()}")
high_salary = [emp for emp in employees if emp.calculate_salary() > 10000]
print(f"\n工资大于10000的员工：")
for emp in high_salary:
    print(f"{emp} - 工资: {emp.calculate_salary()}")
print(f"\n员工对象表示：")
print(repr(employees[0]))
```

解析

57 • 填空 / 简答

编写一个日志分析工具，要求：

1. read_log_file(filename) ：生成器函数，逐行读取日志文件
2. parse_log_line(line) ：解析日志行，提取时间、级别、消息
3. filter_by_level(logs, level) ：生成器函数，按级别过滤日志
4. count_by_level(logs) ：统计各级别日志数量

要求：

- 使用 pathlib 处理文件路径
- 使用生成器实现惰性加载
- 使用装饰器记录函数执行时间
- 处理文件不存在等异常

答案

```
from pathlib import Path
import re
from collections import Counter
import time
from functools import wraps
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} 执行时间: {end-start:.4f}秒")
        return result
    return wrapper
def read_log_file(filename):
    path = Path(filename)
    try:
        if not path.exists():
            raise FileNotFoundError(f"文件{filename} 不存在")
        with path.open('r', encoding='utf-8') as f:
            for line in f:
                yield line.strip()
    except Exception as e:
        print(f"读取文件错误: {e}")
```

```
def parse_log_line(line):
    pattern = r'(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) \[(\w+)\] (.+)'
    match = re.match(pattern, line)
    if match:
        return {
            'time': match.group(1),
            'level': match.group(2),
            'message': match.group(3)
        }
    return None
def filter_by_level(logs, level):
    for log in logs:
        parsed = parse_log_line(log)
        if parsed and parsed['level'] == level:
            yield parsed
@timer
def count_by_level(logs):
    counter = Counter()
    for log in logs:
        parsed = parse_log_line(log)
        if parsed:
            counter[parsed['level']] += 1
    return counter
# 使用示例
log_content = """2024-01-01 10:00:00 [INFO] 系统启动
2024-01-01 10:01:00 [ERROR] 连接失败
2024-01-01 10:02:00 [INFO] 重试连接
2024-01-01 10:03:00 [WARNING] 内存使用过高
2024-01-01 10:04:00 [ERROR] 数据库错误"""
# 创建测试日志文件
log_file = Path('test.log')
log_file.write_text(log_content)
# 分析日志
logs = list(read_log_file('test.log'))
print("所有日志：")
for log in logs:
    print(log)
print("\nERROR级别日志：")
for log in filter_by_level(logs, 'ERROR'):
    print(log)
print("\n日志级别统计：")
stats = count_by_level(logs)
for level, count in stats.items():
    print(f"{level}: {count}")
```

解析

58 • 填空 / 简答

使用Pandas完成以下数据清洗和分析任务：
给定学生成绩数据（包含缺失值和重复数据）：
- 删除重复记录
- 填充缺失值（使用平均分）
- 添加总分和平均分列
- 按平均分排序
- 使用 groupby() 按班级统计
- 使用 apply() 添加等级列（优秀/良好/及格/不及格）
- 使用 pivot_table() 创建数据透视表
- 导出清洗后的数据

答案
```
import pandas as pd
import numpy as np
# 创建包含缺失值和重复数据的DataFrame
data = {
    '姓名': ['张三', '李四', '王五', '张三', '赵六', '钱七'],
    '班级': ['1班', '2班', '1班', '1班', '2班', '1班'],
    '语文': [85, 90, np.nan, 85, 88, 92],
    '数学': [90, 85, 88, 90, np.nan, 95],
    '英语': [88, np.nan, 90, 88, 92, 87]
}
df = pd.DataFrame(data)
print("原始数据：")
print(df)
# 删除重复记录
df_clean = df.drop_duplicates()
print(f"\n删除重复后: {len(df_clean)} 条记录")
# 填充缺失值（使用各科平均分）
for col in ['语文', '数学', '英语']:
    df_clean[col].fillna(df_clean[col].mean(), inplace=True)
# 添加总分和平均分
df_clean['总分'] = df_clean[['语文', '数学', '英语']].sum(axis=1)
df_clean['平均分'] = df_clean[['语文', '数学', '英语']].mean(axis=1)
# 按平均分排序
df_clean = df_clean.sort_values('平均分', ascending=False)
print("\n清洗后的数据：")
print(df_clean)
# 添加等级列
def grade_level(score):
    if score >= 90:
        return '优秀'
    elif score >= 80:
        return '良好'
    elif score >= 60:
        return '及格'
    else:
        return '不及格'
df_clean['等级'] = df_clean['平均分'].apply(grade_level)
```

```
# 按班级统计
class_stats = df_clean.groupby('班级').agg({
    '平均分': ['mean', 'max', 'min'],
    '姓名': 'count'
})
print("\n班级统计：")
print(class_stats)
# 创建数据透视表
pivot = df_clean.pivot_table(
    values='平均分',
    index='班级',
    columns='等级',
    aggfunc='count',
    fill_value=0
)
print("\n数据透视表：")
print(pivot)
# 导出数据
df_clean.to_csv('cleaned_data.csv', index=False, encoding='utf-8-sig')
df_clean.to_excel('cleaned_data.xlsx', index=False)
print("\n数据已导出")
```

解析

59 • 填空 / 简答

编写一个文件组织工具，要求：

1. organize_by_extension(source_dir, target_dir) ：
- 按文件扩展名分类文件
- 创建对应的子目录
- 移动文件到对应目录
2. find_duplicates(directory) ：
- 使用生成器查找重复文件（基于文件大小和内容）
- 返回重复文件列表
3. clean_empty_dirs(directory) ：
- 递归删除空目录

要求：

- 使用 pathlib 进行所有路径操作
- 实现错误处理
- 使用生成器优化内存使用
- 添加日志输出

答案

```
from pathlib import Path
import shutil
import hashlib
def organize_by_extension(source_dir, target_dir):
    source = Path(source_dir)
    target = Path(target_dir)
    target.mkdir(parents=True, exist_ok=True)
    file_count = 0
```

```
    for file in source.rglob('*'):
        if file.is_file():
            ext = file.suffix[1:] if file.suffix else 'no_extension'
            ext_dir = target / ext
            ext_dir.mkdir(exist_ok=True)
            dest_file = ext_dir / file.name
            try:
                shutil.copy2(file, dest_file)
                file_count += 1
                print(f"已复制: {file.name} -> {ext}/")
            except Exception as e:
                print(f"复制失败{file.name}: {e}")
    print(f"\n共组织{file_count} 个文件")
def get_file_hash(file_path):
    hash_md5 = hashlib.md5()
    with open(file_path, 'rb') as f:
        for chunk in iter(lambda: f.read(4096), b""):
            hash_md5.update(chunk)
    return hash_md5.hexdigest()
def find_duplicates(directory):
    path = Path(directory)
    size_map = {}
    # 按大小分组
    for file in path.rglob('*'):
        if file.is_file():
            size = file.stat().st_size
            if size not in size_map:
                size_map[size] = []
            size_map[size].append(file)
    # 检查相同大小的文件
    for size, files in size_map.items():
        if len(files) > 1:
            hash_map = {}
            for file in files:
                file_hash = get_file_hash(file)
                if file_hash not in hash_map:
                hash_map[file_hash] = []
                hash_map[file_hash].append(file)
            for file_hash, dup_files in hash_map.items():
                if len(dup_files) > 1:
                yield dup_files
def clean_empty_dirs(directory):
    path = Path(directory)
    removed_count = 0
    for dir_path in sorted(path.rglob('*'), reverse=True):
        if dir_path.is_dir():
            try:
                if not any(dir_path.iterdir()):
```

```
                dir_path.rmdir()
                removed_count += 1
                print(f"已删除空目录: {dir_path}")
            except Exception as e:
                print(f"删除失败{dir_path}: {e}")
    print(f"\n共删除{removed_count} 个空目录")
# 使用示例
# organize_by_extension('source', 'organized')
# print("\n查找重复文件：")
# for duplicates in find_duplicates('.'):
# print("重复文件组：")
# for file in duplicates:
# print(f" {file}")
# clean_empty_dirs('organized')
```

解析